SYSTEMS & DEVICES 3 (SYS3 - COM00018I) FORMATIVE EXERCISE 1

Computer Architecture Introduction, Computer Design Principles, Pipeline Basics, Structural and Data Hazards.

The questions for the assignment exercises are partly adapted from the book - Computer Architecture: A Quantitative Approach, 5th Edition, John L. Hennessy & David A Patterson

Please try to complete it by 06/03/2026 (Fri, S2 Week 4). We will use the Lecture Review session on 12/03/2026 (Thu, S2 Week 5) for discussions if there are any questions on it.

Q1. Explain the difference between CISC and RISC processors. Name two processors that belong to CISC and RISC.

Solution:

CISC stands for Complex Instruction Set Computer, RISC stands for Reduced Instruction Set Computer.

CISC examples: IBM 370/168, VAX 11/780, Intel x86, etc.

RISC examples: MIPS, ARM, SPARC, ARC, etc.

Q2. Distinguish between a memory-register instruction set (such as x86) and a load-store instruction set (such as MIPS). What are their relative performance advantages in today's technology?

Solution:

Memory-Register Instruction Set (e.g., x86):

Instructions can directly operate on memory operands, without requiring explicit load or store instructions. For example, in x86, instructions like MOV, ADD or SUB can directly access memory locations.

The x86 architecture tends to be **more complex** due to its variable-length instruction encoding and extensive instruction set. It supports a wide range of addressing modes and complex operations.

Performance Advantages: Variable-length instructions can lead to more compact code, reducing memory usage and improving cache performance; x86's extensive instruction set and addressing modes offer **flexibility** in programming and allow for more expressive code; Memory-register architectures can be more **compiler friendly** to compiler optimization and may require less effort from compiler writers to generate efficient code.

Load-Store Instruction Set (e.g., MIPS):

Memory access is strictly controlled through load LW and store SW instructions. Arithmetic and logic instructions only operate on register operands.

Load-store architectures tend to be **simpler and more regular**, with fixed-length instructions and a smaller instruction set. This simplicity can lead to easier pipeline design and better performance predictability.

Performance Advantages: Load-store architectures separate memory access from computation, **reducing memory bandwidth requirements and contention**; The absence of memory-accessing instructions simplifies pipeline design, reducing complexity and potentially **improving clock speed and throughput**; Load-store architectures align well with the principles of Reduced Instruction Set Computing (RISC), focusing on simpler instructions and efficient execution.

Relative Performance Advantages in Today's Technology:

Load-store architectures align well with **modern memory hierarchy** designs, where caches play a crucial role. They tend to exhibit better cache utilization due to the separation of memory operations from computation, potentially leading to improved performance.

Load-store architectures are often favored in **pipelined and superscalar processor** designs due to their simplified instruction semantics, which can facilitate efficient instruction scheduling and execution.

Q3. We wish to compare the performance of two different systems: S1 and S2. System S1 costs £10K and System S2 costs £15K. The following measurements have been made on these systems:

Program	Time on S1	Time on S2
1	10 seconds	5 seconds
2	3 seconds	4 seconds

1) One user cares only about the performance of program 1. Given that cost effectiveness is measured in executions per second per pound, which machine is more cost effective for running program 1? By how much?
2) Another user is concerned with throughput of the systems as measured with an equal workload of programs 1 and 2. Which system has better performance for this workload, given that performance is inversely proportional to execution time? By how much? Which system is more cost effective for this workload? By how much?
3) Yet another user has the following requirements: Program 1 must be executed 200 times each hour. Any remaining time can be used for running program 2. If the system has enough performance to execute program 1 the required number of times per hour, performance is measured by the number of executions per hour for program 2. Which system has higher performance for this workload? Which is more cost effective?

Solution:

1) Executions per second per pound = execution per second / cost = 1 / (execution time x cost)

For S1: 1 / (10 x 10,000) = 1 / 100,000

For S2: 1 / (5 x 15,000) = 1 / 75,000

So S2 is more cost effective by a ratio of 100 / 75 = 1.3 times.

2) The performance is inversely proportional to the time taken for the programs. This is 13 seconds for S1 and 9 seconds for S2. Therefore, S2 outperforms S1 by 13 / 9 = 1.4 times.

The cost effectiveness is computed in the same way as question 1).

For S1: 1/ (13 x 10,000) = 1 / 130,000

For S2: 1/ (9 x 15,000) = 1 / 135,000

So S1 is more cost effective by a ratio of 135 / 130 = 1.04 times.

3) Both machines run P1 the required number of times, the performance is therefore measured by the number of executions per hour of P2.

	Time for 200 executions of P1	Time remaining for P2	Number of executions of P2
S1	2000 seconds	1600 seconds	1600 / 3 = 533
S2	1000 seconds	2600 seconds	2600 / 4 = 650

S2 is faster by 650 / 533 = 1.22 times.

Cost effectiveness in executions per hour per pound:

For S1: 533 / 10,000 = 0.0533

For S2: 650 / 15,000 = 0.043

So S1 is more cost effective by 1.24 times.

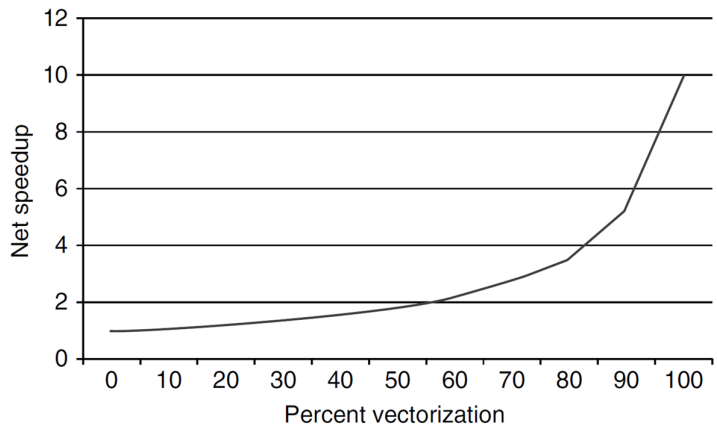
Q4. In this exercise, assume that we are considering enhancing a machine by adding vector hardware to it. When a computation is run in vector mode on the vector hardware, it is 10 times faster than the normal mode of execution. We call the percentage of time that could be spent using vector mode the percentage of vectorization. Vectors are discussed in Chapter 4 of Hennessy's book, but you don't need to know anything about how they work to answer this question!

1) Draw a graph that plots the speedup as a percentage of the computation performed in vector mode. Label the y-axis "Net speedup" and label the x-axis "Percent vectorization."
2) What percentage of vectorization is needed to achieve a speedup of 2?
3) What percentage of vectorization is needed to achieve one-half the maximum speedup attainable from using vector mode?
4) Suppose you have measured the percentage of vectorization of the program to be 70%. The hardware design group estimates it can speed up the vector hardware even more with significant additional investment. You wonder whether the compiler crew could increase the percentage of vectorization, instead. What percentage of vectorization would the compiler team need to achieve in order to equal an addition 2× overall speedup (beyond the initial 10×)?

Solution:

1) See the following plot, which is a plot of the equation

$$y = \frac{100}{(100 - x) + \frac{x}{10}}$$



2) $2 = \frac{1}{(1-x)+\frac{x}{10}}$

X= 0.56 or 56%

3) Maximum speedup = $\frac{1}{\frac{1}{10}}$ = 10

$$5 = \frac{1}{(1-x)+\frac{x}{10}}$$

X= 0.89 or 89%

4) Current speedup: $\frac{1}{0.3+\frac{0.7}{10}}$ = 2.7

Speedup goal: $5.4 = \frac{1}{(1-x)+\frac{x}{10}}$

X = 0.91, which means the percentage of vectorization would need to be 91%.

Q5. Suppose you wish to run a program P with7.5×10^9 instructions on a 5 GHz machine with a CPI of 0.8.

1) What is the expected CPU time?
2) When you run P, it takes 3 seconds of wall clock time to complete. What is the percentage of the CPU time P received?

Solution:

1) CPU Time = clock cycle time × number of instructions × CPI
   = $1/(5 \times 10^9)$ (Sec/cycle) × (7.5×10^9 instructions) × (0.8 cycles/instruction) = 1.2 sec
2) P received 1.2 seconds/3 seconds or 40% of the total CPU time.

Q6. What is it called when the contention for the usage of a hardware device?

Solution:

Structural hazard. The processor contends for the usage of the hardware and might enter into a deadlock state.

Q7. What are the three generic data hazards? What are the differences between them?

Solution:

RAW, WAR and WAW.

Consider two instructions i and j, with i occurring before j.

RAW (read after write) - j tries to read a source before i writes it, so j incorrectly gets the old value.

WAW (write after write) - j tries to write an operand before it is written by i. The writes end up being performed in the wrong order, leaving the value written by i rather than the value written by j in the destination.

WAR (write after read) - j tries to write a destination before it is read by i , so i incorrectly gets the new value.

Q8. Identify all of the data dependencies in the following code. Which dependencies are data hazards that will be resolved via forwarding? Which dependencies are data hazards that will cause a stall?

1 ADD R3, R4, R2

2 SUB R5, R3, R1

3 LW R6, 200(R3)

4 ADD R7, R3, R6

Solution:
There is a data dependency through R3 between the first instruction and each subsequent instruction.
There is a data dependency through R6 between the LW instruction and the last instruction.
The data dependencies between the first instruction and each subsequent instruction can be resolved by using forwarding.
The data dependency between the load and the last add instruction cannot be resolved by using forwarding, cause a stall.